

Java 8 Streams API – Simple Revision Notes

Why Streams were introduced?

Problem before Java 8:

- Collection processing used **for / while loops**
- Code was **long and messy**
- Hard to read & maintain
- Parallel processing was difficult

Solution in Java 8:

Streams API was introduced to process collections in a **clean, functional, and readable way**.

Feature	Description	Example
Stream	Sequence of elements supporting functional-style operations	<code>List<Integer> list = ...; list.stream()</code>
Intermediate Operations	Return a new stream; lazy	<code>filter()</code> , <code>map()</code> , <code>sorted()</code> , <code>distinct()</code>
Terminal Operations	Produce a result or side-effect; triggers execution	<code>forEach()</code> , <code>collect()</code> , <code>count()</code> , <code>reduce()</code>
Parallel Stream	Allows parallel processing	<code>list.parallelStream()</code>
Collectors	Utilities to accumulate results	<code>collect(Collectors.toList())</code> , <code>toSet()</code> , <code>joining()</code>
Key Points	Lazy evaluation, functional style, no storage	stream operations do not modify original collection

❑ What is a Stream?

A **Stream** is a sequence of elements used to **process data** from a collection using **functional style**.

📌 Remember:

- Stream **does not store data**
- Stream **works on collections**
- Stream **does not change original data**

❑ Programming Style Used

Streams follow:

- ☒ **Functional style**
- ☒ **Declarative style**
- ☒ **Not imperative (loop-based)**

Focus on **WHAT to do**, not **HOW to do**.

🔗 Streams depend on these Java 8 features

1. **Lambda Expressions**
2. **Functional Interfaces**
3. **Method References**
4. **Collections Framework**

🔗 Stream Pipeline (MOST IMPORTANT)

A Stream works in **3 steps**:

📄 Source

Collection or array

```
list.stream();
```

🔗 Intermediate Operations

- Process data
- Return another Stream
- Lazy execution

Examples:

- filter()
- map()
- sorted()

```
.filter(n -> n % 2 == 0)
```

🔗 Terminal Operation

- Produces result
- Triggers execution

Examples:

- forEach()
- collect()
- count()

```
.forEach(System.out::println);
```

💎 Simple Example (Must Remember)

```
list.stream()
```

```
.filter(n -> n % 2 == 0)
```

```
.forEach(System.out::println);
```

Important Stream Rules (Non-Negotiable)

✓ Rule 1: Stream does **NOT** store data

Works on existing collection

✓ Rule 2: Stream is **ONE-TIME** use

Cannot reuse a stream

✓ Rule 3: Stream **does NOT** modify original data

Original collection remains unchanged

✓ Rule 4: Without terminal operation → **NO** execution

□ Common Stream Methods (Interview Must-Know)

Intermediate Operations

- filter()
- map()
- sorted()
- distinct()
- limit()

Terminal Operations

- forEach()
- collect()
- count()
- reduce()
- findFirst()

Collectors (Very Common)

Collectors.toList()

Collectors.toSet()

Collectors.groupingBy()

Collectors.counting()

Parallel Streams

`list.parallelStream();`

Use when:

- Large data
- CPU-intensive tasks

Stream vs Collection (Quick Difference)

Stream	Collection
Process data	Store data
One-time use	Reusable
Lazy	Eager
Functional style	OOP style

5 One-Line Interview Answers (Memorize)

1 Streams process collections in functional style

2 Streams do not store data

3 Streams are lazy

4 Streams support parallel processing

5 Streams use lambdas and functional interfaces

Final Memory Formula (Very Easy)

Collection → Stream → filter/map → Result

Final Confidence Note

You now understand:

- WHY Streams exist
- WHAT problem they solved
- HOW they work (basics)
- RULES you must not break

LAMBDA EXPRESSIONS – OVERVIEW

✓ What is a Lambda Expression?

A **lambda expression** is an **anonymous function** used to provide an implementation for a **functional interface**.

Key points:

- No method name
- No access modifier
- Return type is **inferred**
- Short & readable
- Introduced in **Java 8**

□ Lambda Syntax

(parameters) -> expression

Examples:

() -> `System.out.println("Hello");`

(a, b) -> `a + b;`

n -> `n % 2 == 0;`

⚠ Lambda Rules (Non-Negotiable)

1 Works **only with functional interfaces**

2 Cannot be used with normal interfaces

3 Return type is inferred by compiler

4 Body can be single or multiple lines

● Anonymous Class vs Lambda (Quick)

Anonymous Class vs Lambda

Java 7

Java 8

Too much code

Short syntax

Multiple methods

One abstract method

Class based

Function based

🌀 Lambda One-Line Interview Answer

“A lambda expression is an anonymous function introduced in Java 8 to simplify anonymous classes.”

◆ FUNCTIONAL INTERFACES – OVERVIEW

✓ What is a Functional Interface?

An interface that has **exactly ONE** abstract method.

□ Why Functional Interface?

- Lambda needs a **target type**
- Functional interface provides that contract

👉 **Lambda = implementation**

👉 **Functional Interface = rule**

⚠ Functional Interface Rules (Non-Negotiable)

1 Only **ONE** abstract method

2 Can have **default methods**

3 Can have **static methods**

4 @FunctionalInterface is optional but recommended

□ Example

Functional Interface

```
@FunctionalInterface
```

```
interface Greeting {
```

```
    void sayHello();
```

```
}
```

Lambda Implementation

```
Greeting g = () -> System.out.println("Hello");
```

```
g.sayHello();
```

📖 Built-in Functional Interfaces (Very Important)

Interface	Input	Output	Used in
Predicate	1	boolean	filter()
Function<T,R>	1	result	map()
Consumer	1	void	forEach()
Supplier	0	result	get()

❏ Easy Memory Trick

Predicate → condition

Function → convert

Consumer → use

Supplier → give

🔗 How Lambda & Functional Interface Work Together

```
list.stream()
```

```
.filter(n -> n > 10) // Predicate
```

```
.forEach(System.out::println); // Consumer
```

🔗 Functional Interface Interview One-Line

“A functional interface contains exactly one abstract method and is used as the target type for lambda expressions.”

❏ FINAL CLARITY (VERY IMPORTANT)

✗ Lambda is NOT independent

✗ Functional interface is NOT optional

✓ Lambda + Functional Interface **always come together**

📌 FINAL REVISION FORMULA

Anonymous Class → Problem

Lambda → Short Syntax

Functional Interface → Contract

Streams → Uses Lambda + FI

```

1 package streams;
2
3
4 public class PrintallelementsusingForEach {
5
6     public static void main(String[] args) {
7
8
9         // 1Ⓜ Using anonymous inner class
10        Thread t1 = new Thread(new Runnable() {
11            @Override
12            public void run() {
13                System.out.println("Hi from thread without lambda");
14            }
15        });
16        t1.start(); // Start the thread
17
18        // 2Ⓜ Main thread prints
19        System.out.println("Main thread running");
20
21        // 1Ⓜ Example: Thread with lambda
22        Thread t2 = new Thread(() -> {
23            System.out.println("Hi from thread");
24        });
25        t1.start(); // Don't forget to start the thread
26

```

✳️ Anonymous Class vs Lambda – Complete Overview

Feature	Anonymous Class	Lambda Expression
Definition	A class with no name used for one-time implementation	An anonymous function used to implement a functional interface
Reference	Optional. Can be used without reference but cannot reuse	Must have functional interface reference to be executed or reused
Keyword	new keyword required	No new keyword
Syntax	Verbose	Short & concise
Reuse	Only if reference is stored	☑️ Can be reused multiple times via reference
Target Type	Any interface or abstract class	Functional interface only
Java Version	Java 7+	Java 8+
Use Case	One-time object, quick implementation	Cleaner syntax, functional programming, streams

◆ Key Rules You Must Remember

1. **Anonymous class without reference** → can call method **once**.
2. **Lambda** → must be assigned to **functional interface reference** to call **multiple times**.
3. **Lambda cannot implement abstract class**.
4. **Anonymous class can implement abstract class or interface**.
5. **Streams** always use **lambdas** internally (functional style).

◆ Simple Examples Side by Side

Anonymous Class (without reference)

```
new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Hello from anonymous class");  
    }  
}.run(); // ✓ Works once
```

Lambda Expression (with reference)

```
Runnable r = () -> System.out.println("Hello from lambda");  
r.run(); // ✓ Can reuse  
r.run();
```

◆ Interview-Safe One-Liners

1. "Anonymous class can work without a reference but cannot be reused."
2. "Lambda expressions must be assigned to a functional interface and can be reused multiple times."
3. "Anonymous class uses new keyword; lambda does not."
4. "Lambda can only implement functional interfaces; anonymous class can implement abstract class or interface."

□ Mental Model

Think of it like this:

Anonymous Class → old style → verbose, one-time use possible

Lambda → new style → functional, short, reusable with reference

Streams → use lambdas to process collections

PrintallelementsusingForEach.java LambdawithFunctionalInterface1.java AnonymousClassReturnExample.java × Lambda

```

1 package streams;
2 import java.util.*;
3
4 // Employee class representing employee data
5 class Employee1 {
6     String name; // Employee name
7     int salary; // Employee salary
8
9     // Constructor to initialize employee object
10    Employee1(String name, int salary) {
11        this.name = name;
12        this.salary = salary;
13    }
14
15    // Override toString() to print employee details clearly
16    @Override
17    public String toString() {
18        return name + " : " + salary;
19    }
20 }
21
22 public class AnonymousClassReturnExample {
23
24    public static void main(String[] args) {
25
26        // 10 Create a list of employees
27        // Arrays.asList() returns a fixed-size list with 3 employees
28        List<Employee1> employees = Arrays.asList(
29            new Employee1("Alice", 5000),
30            new Employee1("Bob", 7000),
31            new Employee1("Charlie", 6000)
32        );
33
34        // 20 Use an anonymous class as Comparator to find highest-paid employee
35        // Collections.max() needs a Comparator to compare two employees
36        // Anonymous class allows us to implement Comparator inline, without creating a separate class
37        Employee1 highestPaid = Collections.max(employees, new Comparator<Employee1>() {
38            @Override
39            public int compare(Employee1 e1, Employee1 e2) {
40                // Compare employees based on salary
41                // Returns negative if e1 < e2, positive if e1 > e2
42                // Collections.max() uses this logic to find the maximum
43                return e1.salary - e2.salary; // ascending order
44            }
45        });
46
47        // 30 Print the result
48        // The Collections.max() method returns the employee object with max salary
49        System.out.println("Highest Paid Employee: " + highestPaid);
50
51        // Key Notes for Interview:
52        // - Anonymous class is used to implement Comparator on the spot (no separate class)
53        // - Can return values from methods like Collections.max()
54        // - Without reference, the anonymous comparator can still be used because it's passed directly
55        // - With Java 8+, this can be simplified using lambda:
56        // Employee highestPaid = Collections.max(employees, (e1, e2) -> e1.salary - e2.salary);
57    }
58 }
59

```

```
PrintallelementsusingForEach.java  LambdawithFunctionalInterface1.java  AnonymousClassReturnExa
1 package streams;
2 //Lambda is not standalone → Method parameter decides its type
3 import java.util.*;
4
5 // Employee class representing employee data
6 class Employee2 {
7     String name; // Employee name
8     int salary; // Employee salary
9
10    // Constructor to initialize employee object
11    Employee2(String name, int salary) {
12        this.name = name;
13        this.salary = salary;
14    }
15
16    // Override toString() to print employee details
17    @Override
18    public String toString() {
19        return name + " : " + salary;
20    }
21 }
22
23 public class LambdaReturnExample {
24
25    public static void main(String[] args) {
26
27        // 1 Create a list of employees
28        // Arrays.asList() creates a fixed-size list
29        List<Employee2> employees = Arrays.asList(
30            new Employee2("Alice", 5000),
31            new Employee2("Bob", 7000),
32            new Employee2("Charlie", 6000)
33        );
34
35        // 2 Use lambda expression instead of anonymous class
36        // Comparator is a functional interface (only one abstract method: compare)
37        // Lambda replaces the anonymous class implementation
38        Employee2 highestPaid = Collections.max(
39            employees,
40            (e1, e2) -> e1.salary - e2.salary
41        );
42
43        // 3 Print the result
44        // Collections.max() returns the employee with the highest salary
45        System.out.println("Highest Paid Employee: " + highestPaid);
46
47        // Interview Notes:
48        // - Lambda removes boilerplate code of anonymous class
49        // - No 'new' keyword is required
50        // - Lambda is mapped to Comparator's compare() method
51        // - This is modern Java (Java 8+) functional style
52    }
53 }
54
```

Anonymous vs Lambda

What is SAME (70%)

Both:

- Implement **one abstract method**
- Used for **one-time logic**
- Mostly used with:
 - Runnable
 - Comparator
 - ActionListener
- Do not create reusable business logic

❑ Anonymous Class (Java 7)

```
Collections.sort(list, new Comparator<Employee>() {  
    @Override  
    public int compare(Employee e1, Employee e2) {  
        return e1.salary - e2.salary;  
    }  
});
```

✂ Lambda (Java 8)

```
Collections.sort(list, (e1, e2) -> e1.salary - e2.salary);
```

👉 Same logic, fewer lines

👉 That's it. No magic.

❓ Then WHY did Java introduce Lambda?

Because **anonymous classes caused problems**:

✗ Problems with Anonymous

- Too much boilerplate
- Hard to read
- Focus on **HOW**, not **WHAT**
- Nested code becomes ugly

What Lambda Gave

- Cleaner code
- Functional style
- Easier to read
- Enables **Streams API**

🔑 **Streams cannot exist without lambda**

❑ Best Interview Line (USE THIS)

"Lambda expressions are a compact replacement for anonymous classes used with functional interfaces. They reduce boilerplate code but internally work the same way."

Real-Time Usage.

Used mainly in:

- Sorting
- Filtering
- Stream pipelines
- Small utility logic

✗ Not for:

- Core business logic
- Large methods
- Complex flows

You are **NOT** confused now

You just needed **clear WHY** comparison

```
1 package streams;
2
3 @FunctionalInterface
4
5
6 public interface FunctionInterfaceMathOperation {
7
8     public int MathOperation(int a,int b);
9
10 }
11
```

```
1 package streams;
2
3 public class MathOperationDemo {
4
5     public static void main(String[] args) {
6
7         // ♦ Java 8 - Lambda Expression
8         // ♦ Lambda expression
9         // Functional interface reference holds lambda
10        // This reference allows calling the lambda multiple times
11
12        FunctionInterfaceMathOperation multiply =
13            (a, b) -> a * b;
14
15        FunctionInterfaceMathOperation add =
16            (a, b) -> a + b;
17
18
19        System.out.println("Lambda Add: " +
20            add.MathOperation(10, 20));
21
22        System.out.println("Lambda Multiply: " +
23            multiply.MathOperation(10, 20));
24    }
25 }
26
27
```

◆ Predicate<T>

👉 Used for CONDITIONS (true / false)

Logic

- Takes **one input**
- Returns **boolean**
- Used in **filtering**

Method

boolean test(T t);

```
PredicateExample.java ×
1 package streams;
2
3 import java.util.function.Predicate;
4
5 public class PredicateExample {
6     public static void main(String[] args) {
7
8         Predicate<Integer> isEven =
9             n -> n % 2 == 0;
10
11         System.out.println(isEven.test(10)); // true
12         System.out.println(isEven.test(7));  // false
13     }
14 }
15
16
17
18
```

Usage

- filter()
- Validation
- Conditions

☐ **Memory:** Predicate → Question? ✓ / ✗

◆ **Function<T, R>**

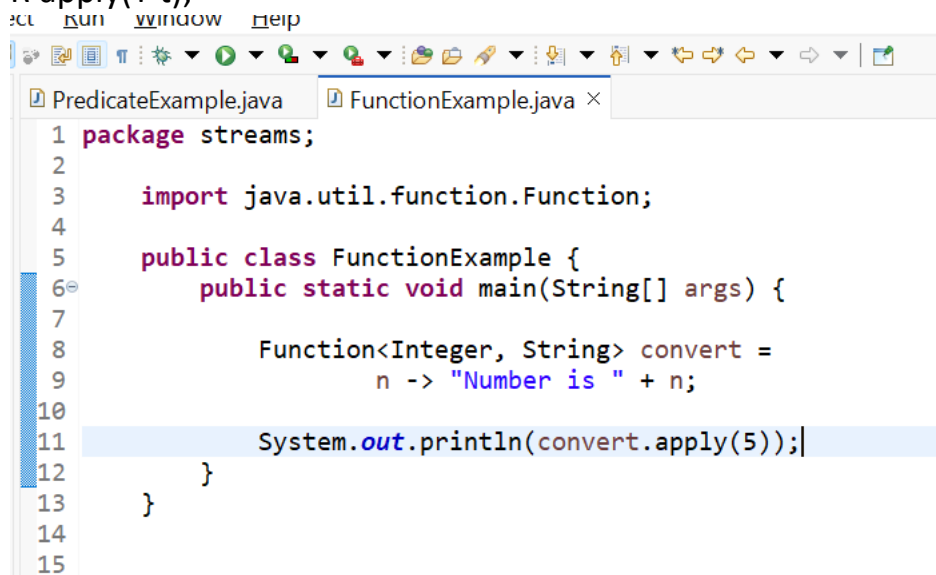
👉 **Used for TRANSFORMATION**

Logic

- Takes **one input**
- Returns **one output**

Method

R apply(T t);



```
1 package streams;
2
3 import java.util.function.Function;
4
5 public class FunctionExample {
6     public static void main(String[] args) {
7
8         Function<Integer, String> convert =
9             n -> "Number is " + n;
10
11         System.out.println(convert.apply(5));
12     }
13 }
14
15
```

Usage

- map()
- Conversions
- Data transformation

☐ **Memory:** Function → Input → Output

❖ Consumer<T>

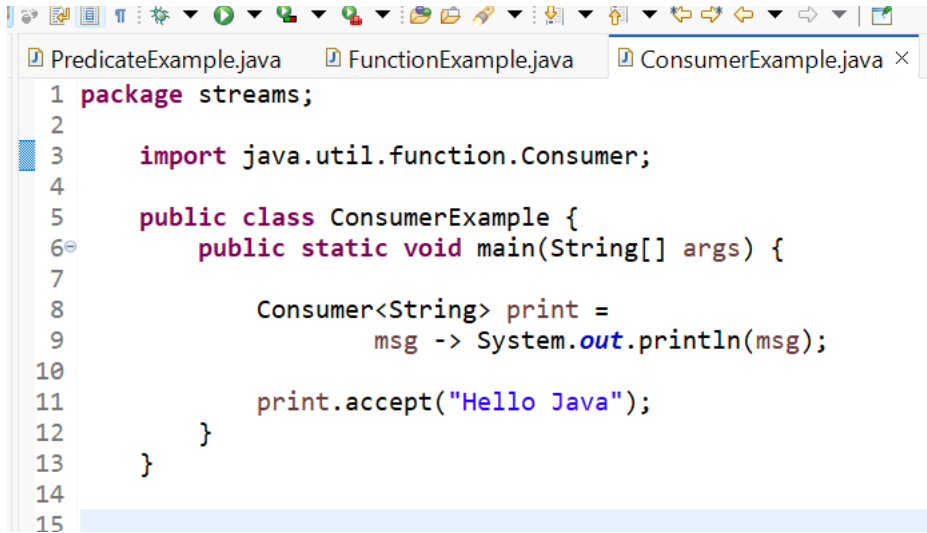
👉 Used for DOING ACTION (no return)

Logic

- Takes **one input**
- Returns **nothing**

Method

void accept(T t);



```
1 package streams;
2
3 import java.util.function.Consumer;
4
5 public class ConsumerExample {
6     public static void main(String[] args) {
7
8         Consumer<String> print =
9             msg -> System.out.println(msg);
10
11         print.accept("Hello Java");
12     }
13 }
14
15
```

Usage

- forEach()
- Printing
- Logging

☐ **Memory:** Consumer → Consumes & finishes

Supplier<T>

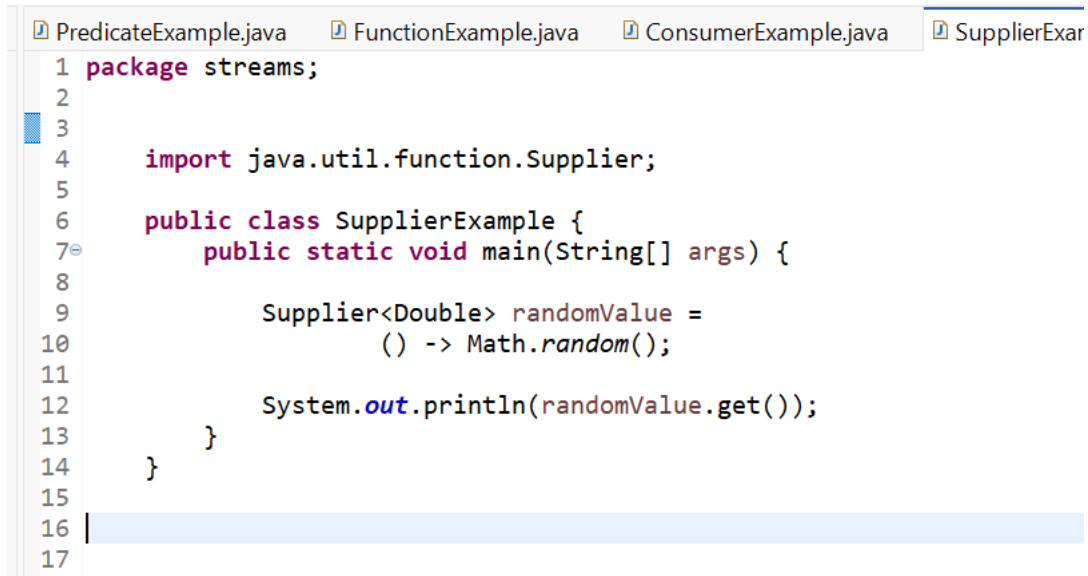
👉 Used for PROVIDING data

Logic

- Takes **no input**
- Returns **output**

Method

T get();



The screenshot shows a Java IDE with four tabs: PredicateExample.java, FunctionExample.java, ConsumerExample.java, and SupplierExample.java. The SupplierExample.java tab is active, displaying the following code:

```
1 package streams;
2
3
4 import java.util.function.Supplier;
5
6 public class SupplierExample {
7     public static void main(String[] args) {
8
9         Supplier<Double> randomValue =
10             () -> Math.random();
11
12         System.out.println(randomValue.get());
13     }
14 }
15
16
17
```

Usage

- Object creation
- Lazy values
- Default values

☐ **Memory:** Supplier → Gives data

Streams Method Functional Interface

filter()	Predicate
map()	Function
forEach()	Consumer
generate()	Supplier

Lines:

- **Predicate** → Condition checking
- **Function** → Data transformation
- **Consumer** → Action without return
- **Supplier** → Provides data

❖ **BiPredicate<T, U>**

👉 **Condition with TWO inputs**

Logic

- Takes **two inputs**
- Returns **boolean**
- Used for **checking conditions**

Method

boolean test(T t, U u);

```
PredicateExample.java  FunctionExample.java  ConsumerExample.java  SupplierExample.java  BiPredicateExample
1 package streams;
2
3 import java.util.function.BiPredicate;
4
5 public class BiPredicateExample {
6     public static void main(String[] args) {
7
8         // Check if sum of two numbers is greater than 50
9         BiPredicate<Integer, Integer> isGreater =
10             (a, b) -> (a + b) > 50;
11
12         System.out.println(isGreater.test(20, 40)); // true
13         System.out.println(isGreater.test(10, 20)); // false
14     }
15 }
16
17
```

Real-Time Usage

- Validation with two fields
- Comparing two values

☐ **Memory:** BiPredicate → 2 inputs → true/false

❖ **BiFunction<T, U, R>**

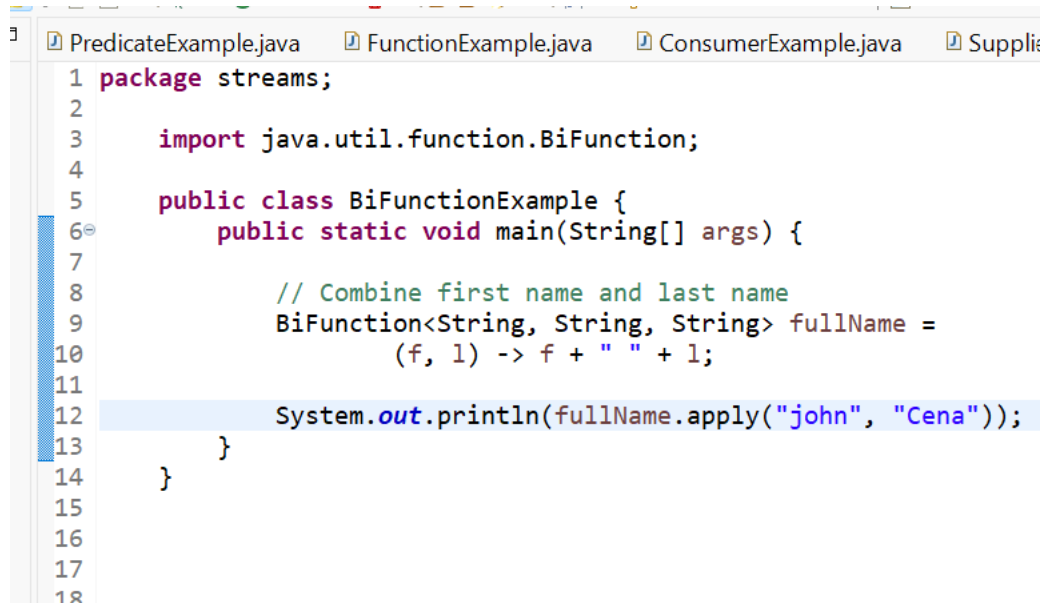
📖 **Combine two inputs → produce result**

Logic

- Takes **two inputs**
- Returns **one result**

Method

R apply(T t, U u);

A screenshot of a Java IDE with a tab titled 'BiFunctionExample.java'. The code is as follows:

```
1 package streams;
2
3 import java.util.function.BiFunction;
4
5 public class BiFunctionExample {
6     public static void main(String[] args) {
7
8         // Combine first name and last name
9         BiFunction<String, String, String> fullName =
10             (f, l) -> f + " " + l;
11
12         System.out.println(fullName.apply("john", "Cena"));
13     }
14 }
15
16
17
18
```

Real-Time Usage

- Calculations
- Combining values
- Mapping two fields to one

☐ **Memory:** BiFunction → 2 inputs → 1 output

❖ BiConsumer<T, U>

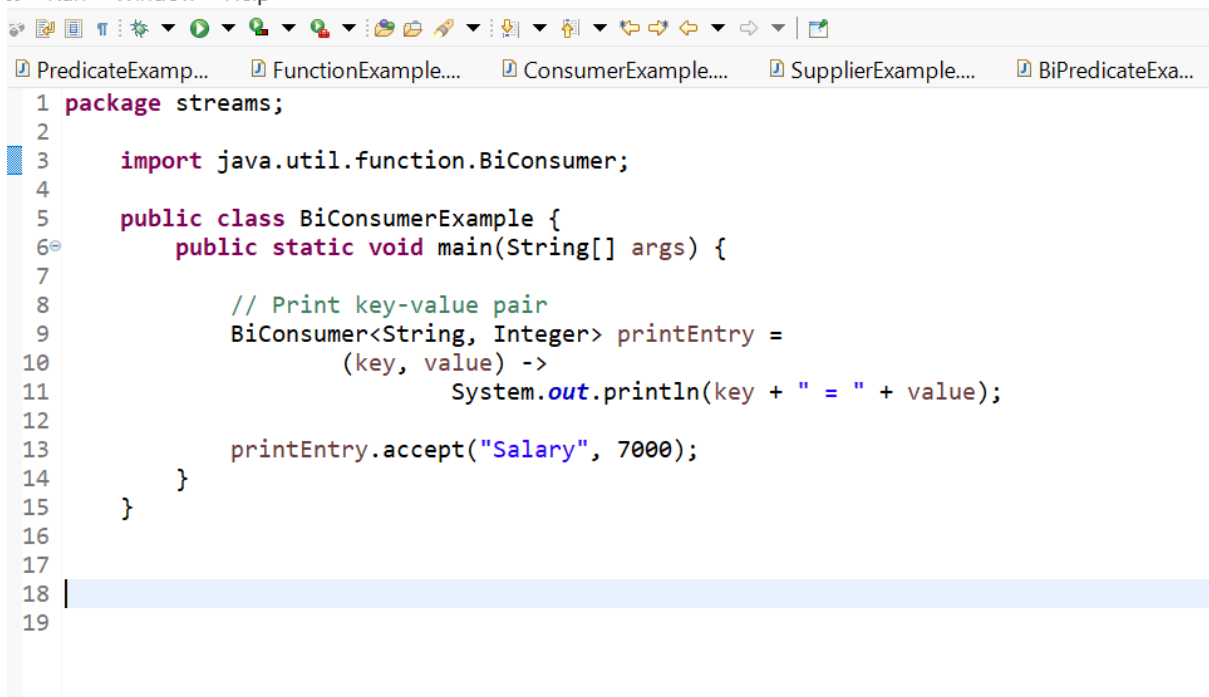
👉 Perform action on two inputs (no return)

Logic

- Takes **two inputs**
- Returns **nothing**

Method

void accept(T t, U u);

A screenshot of an IDE window showing a Java code file named 'ConsumerExample...'. The code is as follows:

```
1 package streams;
2
3 import java.util.function.BiConsumer;
4
5 public class BiConsumerExample {
6     public static void main(String[] args) {
7
8         // Print key-value pair
9         BiConsumer<String, Integer> printEntry =
10             (key, value) ->
11                 System.out.println(key + " = " + value);
12
13         printEntry.accept("Salary", 7000);
14     }
15 }
16
17
18
19
```

Real-Time Usage

- Printing
- Logging
- Map iteration

☐ **Memory:** BiConsumer → 2 inputs → action

✓ Java Streams – Answers & Examples

◆ Basic Level (1–15)

1. Create a stream from a list

```
List<Integer> list = List.of(1,2,3);  
Stream<Integer> stream = list.stream();
```

2. Convert a stream back to a list

```
List<Integer> result = stream.collect(Collectors.toList());
```

3. Count elements in a stream

```
long count = list.stream().count();
```

4. Filter even numbers

```
list.stream()  
    .filter(n -> n % 2 == 0)  
    .toList();
```

5. Convert strings to uppercase

```
list.stream()  
    .map(String::toUpperCase)  
    .toList();
```

6. Sort a list

```
list.stream().sorted().toList();
```

7. Limit first 5 elements

```
list.stream().limit(5).toList();
```

8. Skip first 3 elements

```
list.stream().skip(3).toList();
```

9. Find first element

```
Optional<Integer> first = list.stream().findFirst();
```

10. Check if all match condition

```
boolean allEven = list.stream().allMatch(n -> n % 2 == 0);
```

11. Check if any match condition

```
boolean anyEven = list.stream().anyMatch(n -> n % 2 == 0);
```

12. Remove duplicates

```
list.stream().distinct().toList();
```

13. Collect into Set

```
Set<Integer> set = list.stream().collect(Collectors.toSet());
```

14. Infinite stream of random numbers

```
Stream.generate(Math::random).limit(5).toList();
```

15. Create stream from array

```
Arrays.stream(arr);
```

◆ Intermediate Level

16. Sum of integers

```
int sum = list.stream().mapToInt(Integer::intValue).sum();
```

17. Max & Min

```
int max = list.stream().max(Integer::compare).get();
```

```
int min = list.stream().min(Integer::compare).get();
```

18. Average

```
double avg = list.stream().mapToInt(i -> i).average().getAsDouble();
```

19. Concatenate multiple lists

```
Stream.concat(list1.stream(), list2.stream()).toList();
```

20. Group even & odd

```
Map<Boolean, List<Integer>> map =
```

```
list.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

21. Group employees by department

```
employees.stream()
```

```
.collect(Collectors.groupingBy(Employee::getDept));
```

22. Second highest number

```
list.stream()
```

```
.distinct()
```

```
.sorted(Comparator.reverseOrder())
```

```
.skip(1)
```

```
.findFirst();
```

23. Partition >10 and ≤10

```
Collectors.partitioningBy(n -> n > 10);
```

24. Word frequency

```
list.stream()
```

```
.collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

25. Remove null values

```
list.stream().filter(Objects::nonNull).toList();
```

26. Concatenate strings

```
list.stream().collect(Collectors.joining());
```

27. Flatten list of lists

```
listOfLists.stream()
```

```
.flatMap(List::stream)
```

```
.toList();
```

28. Find duplicate elements

```
Set<Integer> seen = new HashSet<>();  
list.stream().filter(n -> !seen.add(n)).toList();
```

29. Top 3 highest numbers

```
list.stream()  
.sorted(Comparator.reverseOrder())  
.limit(3)  
.toList();
```

30. String → length map

```
list.stream()  
.collect(Collectors.toMap(s -> s, String::length));
```

31. Longest word

```
list.stream()  
.max(Comparator.comparingInt(String::length));
```

32. Distinct characters

```
list.stream()  
.flatMap(s -> s.chars().mapToObj(c -> (char) c))  
.distinct()  
.toList();
```

33. Merge two maps

```
map2.forEach((k,v) -> map1.merge(k, v, Integer::sum));
```

34. Parallel stream

```
list.parallelStream().forEach(System.out::println);
```

35. Summary report

```
IntSummaryStatistics stats =  
list.stream().mapToInt(i -> i).summaryStatistics();
```

Important

- Streams are **lazy**
- **Intermediate ops** → filter, map, sorted
- **Terminal ops** → collect, forEach, reduce
- Prefer **method references** when possible
- Avoid modifying external variables in streams

◆ Method Reference

A **method reference** is a **shortcut for a lambda expression** that calls an **existing method**.

Syntax

ClassName::methodName

objectName::methodName

Example

```
list.forEach(System.out::println);
```

✓ Shorter

✓ More readable

◆ Types of Method Reference

▣ Static Method Reference

Math::max

▣ Instance Method Reference

obj::methodName

▣ Instance Method of Arbitrary Object

String::toUpperCase

◆ Constructor Reference

A **constructor reference** is used to **create objects** using a constructor.

It is a special form of method reference.

Syntax

ClassName::new

Example

```
Supplier<List<String>> s = ArrayList::new;
```

```
List<String> list = s.get();
```

Parameterized Constructor Example

```
Function<String, StringBuilder> f = StringBuilder::new;
```

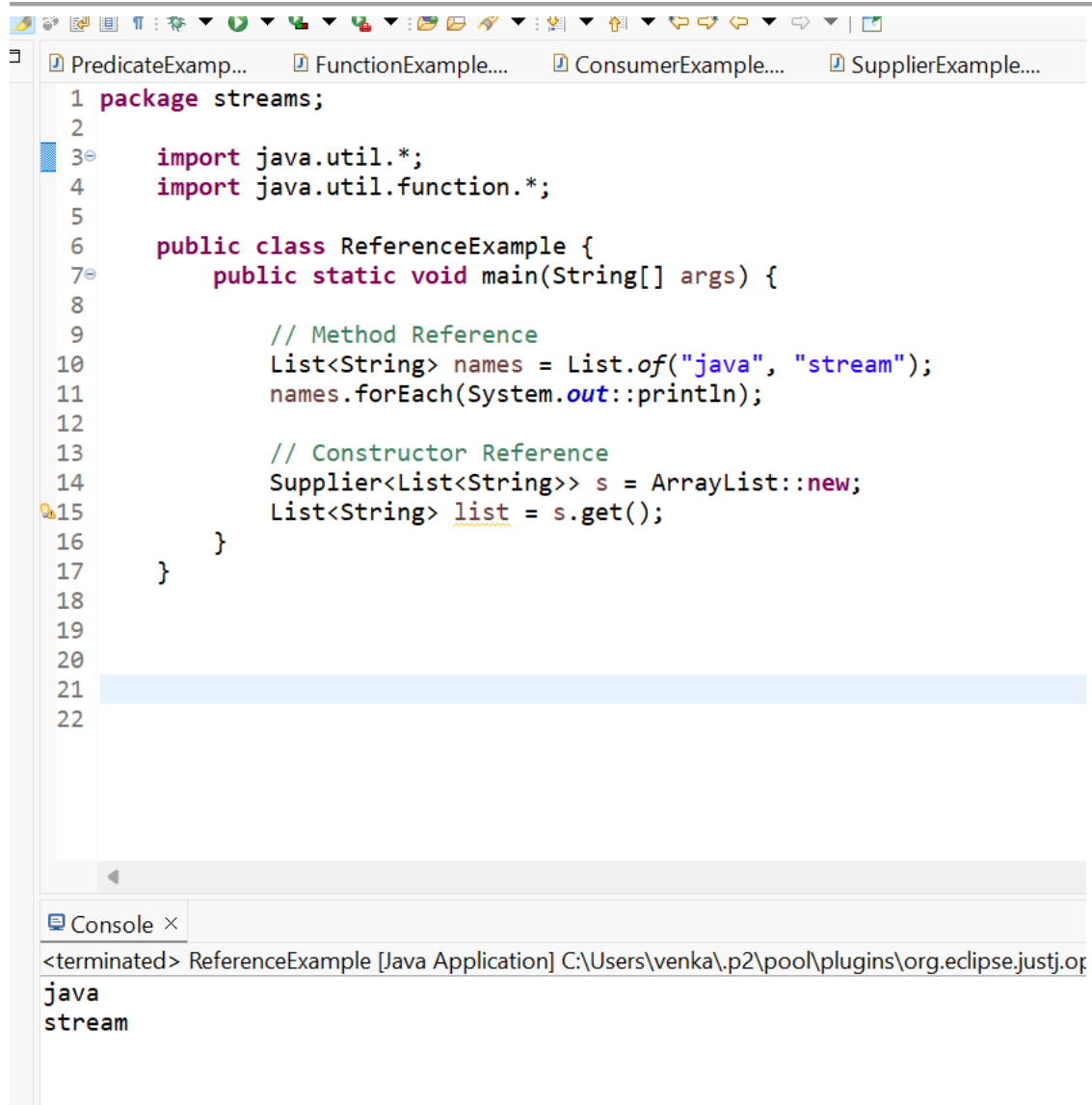
```
StringBuilder sb = f.apply("Hello");
```

🔑 Difference (Easy Table)

Feature	Method Reference	Constructor Reference
Purpose	Call existing method	Create object
Symbol	::methodName	::new
	Replaces Lambda calling method	Lambda using new

Interview One-Line Answer

Method reference is a shorthand of lambda expression to call an existing method, while constructor reference is used to create objects.



```
1 package streams;
2
3 import java.util.*;
4 import java.util.function.*;
5
6 public class ReferenceExample {
7     public static void main(String[] args) {
8
9         // Method Reference
10        List<String> names = List.of("java", "stream");
11        names.forEach(System.out::println);
12
13        // Constructor Reference
14        Supplier<List<String>> s = ArrayList::new;
15        List<String> list = s.get();
16    }
17 }
18
19
20
21
22
```

Console ×

```
<terminated> ReferenceExample [Java Application] C:\Users\venka\.p2\pool\plugins\org.eclipse.justj.o
java
stream
```

1 abstract method → Lambda / Method Reference

2+ abstract methods → NOT allowed

```
Employee.java x BiConsumerExample.java
1 package streams;
2
3 import java.util.List;
4 import java.util.Set;
5 import java.util.stream.Collectors;
6
7 public class Employee {
8     int id;
9     String name;
10    String email;
11    boolean active;
12
13    public Employee(int id, String name, String email, boolean active) {
14        this.id = id;
15        this.name = name;
16        this.email = email;
17        this.active = active;
18    }
19
20    boolean isActive() {
21        return active;
22    }
23
24    String getEmail() {
25        return email;
26    }
27
28    @Override
29    public String toString() {
30        return "Employee [id=" + id + ", name=" + name + ", email=" + email + ", active=" + active + "]";
31    }
32
33    public static void main(String[] args) {
34
35        List<Employee> employees = List.of(
36            new Employee(1, "hari", "Hari@Company.com", true),
37            new Employee(2, "Ravi", "ravi@company.com", false),
38            new Employee(3, "Anu", "ANU@Company.com", true)
39        );
40
41        Set<String> onboardingEmails =
42            employees.stream()
43                .filter(Employee::isActive) // remove inactive
44                .map(e -> e.getEmail().toLowerCase()) // normalize
45                .collect(Collectors.toSet());
46
47        System.out.println(onboardingEmails);
48    }
49 }
50 }
51 }
```